Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет им. И.Н. Ульянова»
Кафедра вычислительной техники

Лабораторная работа № 8

Чтение файла параметров

По дисциплине «Операционные системы»

Выполнил: студент группы ИВТ-42-23
Иванов И.Л.

Проверил: кандидат технических наук
Ковалев С.В.

Чебоксары, 2026

Цель работы

Реализовать чтение входных параметров модели ОС из JSON-файла, генерацию значений по диапазону и дискретности, а также повторяемость последовательности псевдослучайных чисел.
Исходники: https://github.com/kukirmash/os_model/

Задание

- разработать структуру файла параметров;
- добавить типы и данные для хранения диапазонов, дискретности и комментариев;
- реализовать подпрограммы «Генерация значения параметра», «Генерация параметров системы» и «Чтение файла параметров»;
- обновить подпрограмму «Генерация нового задания»;
- обеспечить чтение файла один раз при запуске и повторяемость результатов.

В отчёте представить блок-схемы алгоритмов, анализ программного кода, листинг программы и экранные формы.

Краткая характеристика решения

Параметры вынесены в JSON-файл params.json. Для каждого параметра задаются min, max, step и comment. Файл читается один раз в точке входа приложения; System.Init только генерирует системные значения по уже загруженным диапазонам. Для повторяемости используется один генератор math/rand/v2 с фиксированным зерном, общий для параметров, заданий и команд процессора.

1. Структура файла параметров

Файл params.json содержит три верхнеуровневых раздела: seed, system и task. Каждый параметр описывается объектом с полями min, max, step и comment.

Параметр	Мин	Макс	Шаг
Объём памяти, единиц	4096	8192	1024
Длительность такта, мс	10	100	10
Максимальная длительность ВВ, тактов	1	10	1
Затраты на загрузку, тактов	1	1	1
Затраты на переключение, тактов	1	1	1
Затраты на инициализацию ВВ, тактов	1	1	1
Затраты на прерывание ВВ, тактов	1	1	1
Затраты на завершение, тактов	1	1	1
Размер задания, единиц памяти	100	500	50
Число команд задания	10	20	1
Процент команд ВВ, %	0	50	5
Базовый приоритет задания	1	10	1

Пример файла:


```
{
 "seed": 2026,
 "system": {
  "memorySize": {
   "min": 4096,
   "max": 8192,
   "step": 1024,
   "comment": "Объём памяти, единиц"
  },
  "tickDurationMs": {
   "min": 10,
   "max": 100,
   "step": 10,
   "comment": "Длительность такта, мс"
  },
  "ioMaxDuration": {
   "min": 1,
   "max": 10,
   "step": 1,
   "comment": "Максимальная длительность ВВ, тактов"
  },
  "costs": {
   "load": {
    "min": 1,
    "max": 1,
    "step": 1,
    "comment": "Затраты на загрузку задания, тактов"
   },
   "dispatch": {
    "min": 1,
    "max": 1,
    "step": 1,
    "comment": "Затраты на выбор процесса, тактов"
   },
   "ioInit": {
    "min": 1,
    "max": 1,
    "step": 1,
    "comment": "Затраты на инициализацию ВВ, тактов"
```

```
    },
    "ioInterrupt": {
     "min": 1,
     "max": 1,
     "step": 1,
     "comment": "Затраты на обслуживание прерывания ВВ, тактов"
    },
    "terminate": {
     "min": 1,
     "max": 1,
     "step": 1,
     "comment": "Затраты на завершение задания, тактов"
    }
   }
  },
  "task": {
   "size": {
    "min": 100,
    "max": 500,
    "step": 50,
    "comment": "Размер задания, единиц памяти"
   },
   "commands": {
    "min": 10,
    "max": 20,
    "step": 1,
    "comment": "Число команд задания"
   },
   "ioPercent": {
    "min": 0,
    "max": 50,
    "step": 5,
    "comment": "Процент команд ВВ, %"
   },
   "basePriority": {
    "min": 1,
    "max": 10,
    "step": 1,
    "comment": "Базовый приоритет задания"
   }
  }
}
```

2. Типы данных и алгоритмы

В пакете internal/params определены типы Param, System, Task, Set, SystemValues и TaskValues. Param хранит диапазон и дискретность, Set — полный набор параметров из файла, SystemValues и TaskValues — сгенерированные значения.

```
type Param struct {
    Min, Max, Step int
    Comment        string
}

type Set struct {
    Seed   uint64
    System System
```

```
    Task   Task
}

type SystemValues struct {
    MemorySize, TickDurationMs, IOMaxDuration int
    Costs CostsValues
}
```

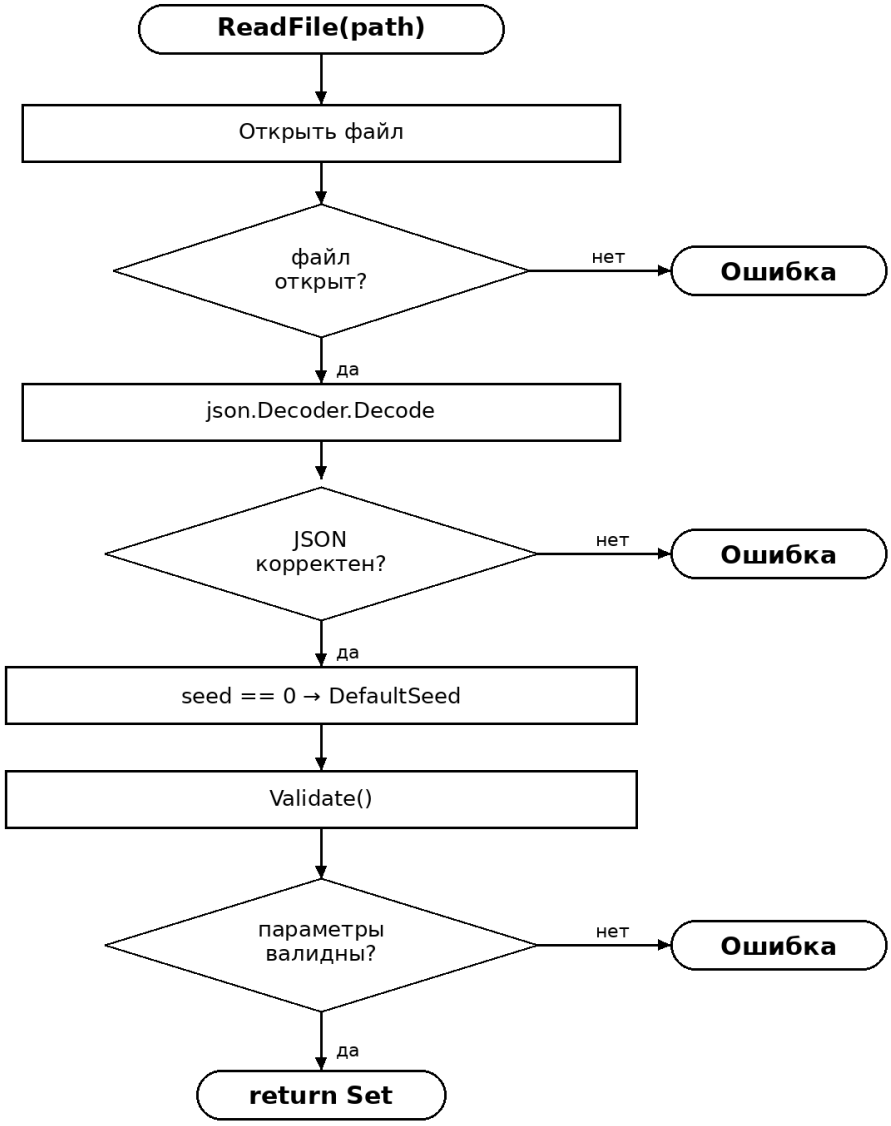


Рисунок 1 — Блок-схема чтения файла параметров

ReadFile открывает JSON-файл, декодирует его с запретом неизвестных полей, при необходимости подставляет DefaultSeed и проверяет корректность всех диапазонов. Ошибки содержат имя параметра и причину.

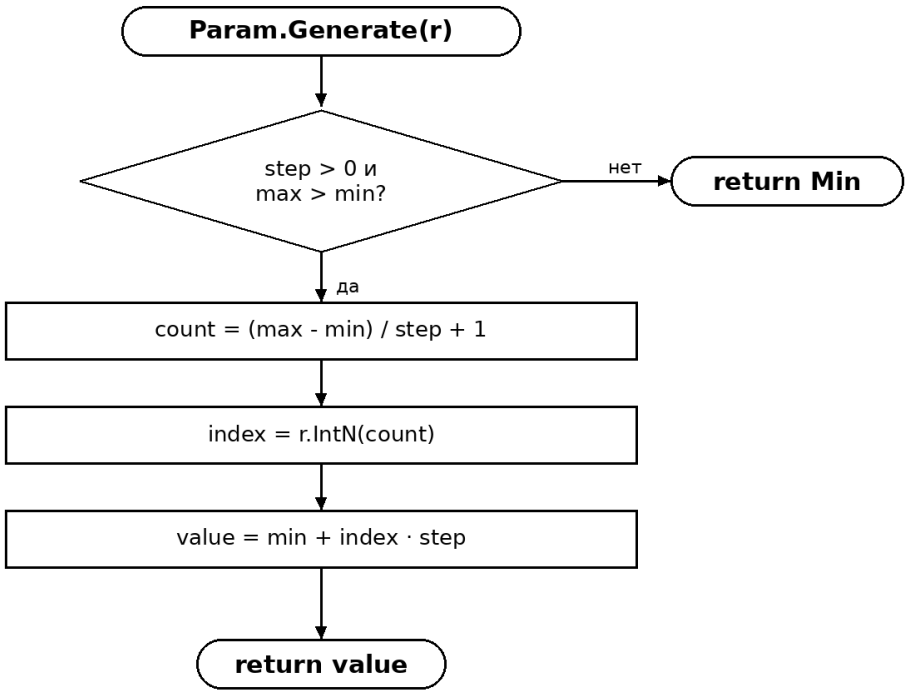


Рисунок 2 — Блок-схема генерации значения параметра

Генерация значения учитывает дискретность: количество допустимых значений вычисляется как (max − min) / step + 1, после чего выбирается случайный индекс. Это гарантирует, что значение не выйдет за диапазон и будет кратно шагу.

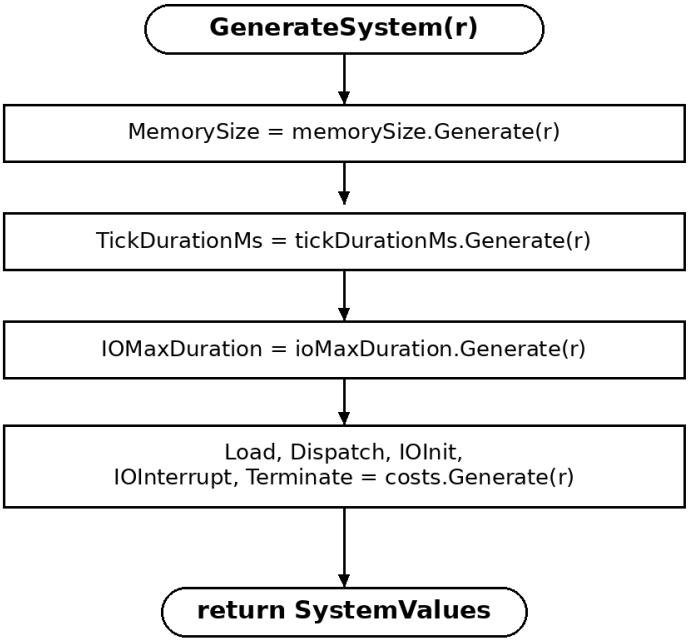


Рисунок 3 — Блок-схема генерации параметров системы

GenerateSystem последовательно генерирует объём памяти, длительность такта, максимальную длительность BB и пять длительностей системных действий. Результат преобразуется в Config и regulator.Costs.

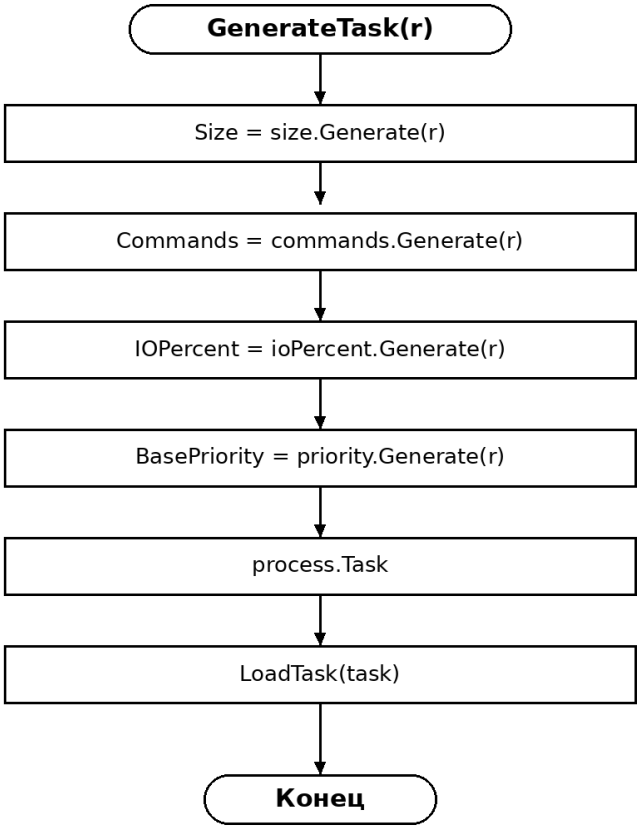


Рисунок 4 — Блок-схема генерации нового задания

GenerateTask генерирует размер задания, число команд, процент команд BB и базовый приоритет. Полученные значения передаются в process.Task и затем в LoadTask.

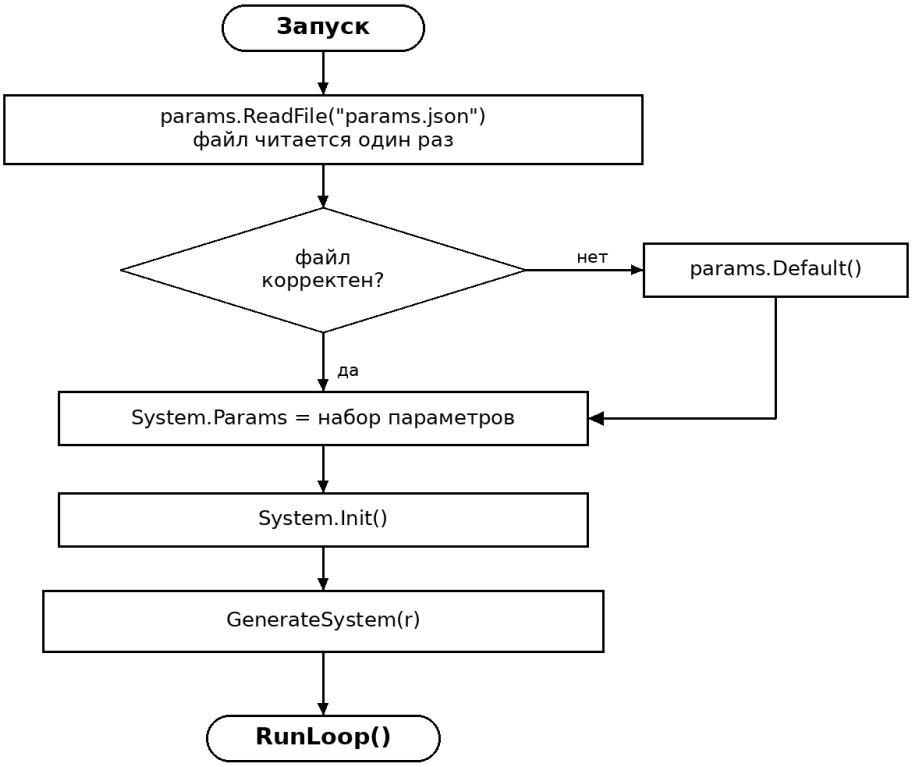


Рисунок 5 — Общая схема запуска с чтением файла параметров

Файл читается один раз в app.startup или cmd/osmodel. При ошибке Wails-приложение использует params.Default(), а причина отображается в интерфейсе. System.Init не открывает файл повторно.

3. Анализ программного кода

3.1. Пакет params

Пакет params не зависит от ядра модели. Он отвечает только за хранение диапазонов, чтение JSON и генерацию значений. Это позволяет использовать его в консольном режиме, Wails-приложении и тестах без циклических зависимостей.

internal/params/read.go

```
package params

import (
	"encoding/json"
	"fmt"
	"os"
)

// ---------------------------------------------------------------------------
// ReadFile читает JSON-файл параметров. Файл должен читаться один раз при
// запуске модели; повторное чтение при генерации заданий не выполняется.
func ReadFile(path string) (Set, error) {
	file, err := os.Open(path)
	if err != nil {
		return Set{}, fmt.Errorf("открыть файл параметров %q: %w", path, err)
	}
	defer file.Close()

	decoder := json.NewDecoder(file)
	decoder.DisallowUnknownFields()

	var set Set
	if err := decoder.Decode(&set); err != nil {
		return Set{}, fmt.Errorf("разобрать файл параметров %q: %w", path, err)
	}

	if set.Seed == 0 {
		set.Seed = DefaultSeed
	}
	if err := set.Validate(); err != nil {
		return Set{}, fmt.Errorf("некорректные параметры в %q: %w", path, err)
	}

	return set, nil
}
```

3.2. Генерация значений

Метод Param.Generate не использует глобальный генератор: ему передаётся *rand.Rand. Если шаг некорректен или диапазон вырожден, возвращается минимальное значение.

internal/params/generate.go

```
package params

import "math/rand/v2"

// ---------------------------------------------------------------------------
// Generate — подпрограмма «Генерация значения параметра».
// Возвращает случайное значение из диапазона [Min, Max] с шагом Step.
func (p Param) Generate(r *rand.Rand) int {
	if p.Step <= 0 || p.Max <= p.Min || r == nil {
		return p.Min
	}

	count := (p.Max-p.Min)/p.Step + 1
	if count <= 0 {
		return p.Min
	}
	return p.Min + r.IntN(count)*p.Step
}

// ---------------------------------------------------------------------------
// GenerateSystem — подпрограмма «Генерация параметров системы».
// Генерирует фактические значения системных параметров по диапазонам.
func (s Set) GenerateSystem(r *rand.Rand) SystemValues {
	return SystemValues{
		MemorySize:     s.System.MemorySize.Generate(r),
		TickDurationMs: s.System.TickDurationMs.Generate(r),
		IOMaxDuration:  s.System.IOMaxDuration.Generate(r),
		Costs: CostsValues{
			Load:       s.System.Costs.Load.Generate(r),
			Dispatch:   s.System.Costs.Dispatch.Generate(r),
			IOInit:     s.System.Costs.IOInit.Generate(r),
			IOInterrupt: s.System.Costs.IOInterrupt.Generate(r),
			Terminate:  s.System.Costs.Terminate.Generate(r),
		},
	}
}

// ---------------------------------------------------------------------------
// GenerateTask — обновлённая подпрограмма «Генерация нового задания».
// Генерирует параметры задания по диапазонам из файла.
func (t Task) GenerateTask(r *rand.Rand) TaskValues {
	return TaskValues{
		Size:        t.Size.Generate(r),
		Commands:    t.Commands.Generate(r),
		IOPercent:   t.IOPercent.Generate(r),
		BasePriority: t.BasePriority.Generate(r),
	}
}
```

3.3. Интеграция в System

В System добавлены Params, Rand, ParamsPath, ParamsLoaded и ParamsError. При Init параметры по умолчанию подставляются только при отсутствии файла; явно заданные поля Config сохраняют приоритет, что важно для старых лабораторных работ.

```
if sys.Params.IsZero() {
   sys.Params = params.Default()
```

```
}
if sys.Rand == nil {
   seed := sys.Params.Seed
   sys.Rand = rand.New(rand.NewPCG(seed, seed))
}

values := sys.Params.GenerateSystem(sys.Rand)
if sys.Config.MemorySize <= 0 {
   sys.Config.MemorySize = values.MemorySize
}
if sys.Config.TickDurationMs <= 0 {
   sys.Config.TickDurationMs = values.TickDurationMs
}
if sys.Config.IOMaxDuration <= 0 {
   sys.Config.IOMaxDuration = values.IOMaxDuration
}
```

Единый генератор передаётся процессору: CPU.Rand. Все вызовы rand.IntN в CPU заменены на cpu.randomIntN, поэтому типы команд, адреса операндов и длительности BB также воспроизводимы.

```
type CPU struct {
   // ...
   Rand *rand.Rand
}

func (cpu *CPU) randomIntN(n int) int {
   if n <= 0 {
      return 0
   }
   if cpu.Rand != nil {
      return cpu.Rand.IntN(n)
   }
   return rand.IntN(n)
}
```

3.4. Точки входа и интерфейс

Wails-приложение читает файл в startup, консольный режим — по флагу -params. Путь можно задать переменной OS_MODEL_PARAMS. При ошибке приложение не падает: используются значения по умолчанию, а текст ошибки выводится в панели «Параметры из JSON-файла».

```
path := stdos.Getenv("OS_MODEL_PARAMS")
if path == "" {
   path = params.DefaultPath
}
set, err := params.ReadFile(path)
loaded := err == nil
if err != nil {
   set = params.Default()
}
```

В снимок состояния добавлены сведения о файле, seed и таблица диапазонов. Панель интерфейса показывает имя файла, статус загрузки, seed

и все параметры с минимумом, максимумом и шагом. В «Параметрах нового задания» диапазоны выводятся вместе с дискретностью.

4. Листинг основных модулей

Ниже приведены полные листинги новых модулей пакета params. Полный листинг проекта доступен в репозитории.

internal/params/params.go

```
// Package params хранит входные параметры модели ОС и правила их генерации
// (лабораторная работа 8).
//
// Параметры читаются из JSON-файла один раз при запуске модели. Для каждого
// параметра задаются минимум, максимум, дискретность и комментарий.
package params

import "fmt"

// DefaultPath — имя файла параметров по умолчанию.
const DefaultPath = "params.json"

// DefaultSeed — фиксированное зерно генератора псевдослучайных чисел.
// Одинаковое зерно обеспечивает повторяемость результатов при каждом запуске.
const DefaultSeed uint64 = 2026

// ---------------------------------------------------------------------------
// Param — диапазон и дискретность одного входного параметра.
type Param struct {
	Min     int    `json:"min"`     // минимальное значение
	Max     int    `json:"max"`     // максимальное значение
	Step    int    `json:"step"`    // дискретность
	Comment string `json:"comment"` // комментарий для индикации
}

// ---------------------------------------------------------------------------
// Costs — диапазоны длительностей системных действий.
type Costs struct {
	Load       Param `json:"load"`
	Dispatch   Param `json:"dispatch"`
	IOInit     Param `json:"ioInit"`
	IOInterrupt Param `json:"ioInterrupt"`
	Terminate  Param `json:"terminate"`
}

// ---------------------------------------------------------------------------
// System — диапазоны системных параметров.
type System struct {
	MemorySize    Param `json:"memorySize"`
	TickDurationMs Param `json:"tickDurationMs"`
	IOMaxDuration  Param `json:"ioMaxDuration"`
	Costs         Costs `json:"costs"`
}

// ---------------------------------------------------------------------------
// Task — диапазоны параметров генерируемых заданий.
type Task struct {
	Size        Param `json:"size"`
	Commands    Param `json:"commands"`
	IOPercent   Param `json:"ioPercent"`
	BasePriority Param `json:"basePriority"`
}

// ---------------------------------------------------------------------------
```

```
// Set — полный набор параметров из файла.
type Set struct {
	Seed   uint64 `json:"seed"`   // зерно генератора (0 — значение по умолчанию)
	System System `json:"system"` // системные параметры
	Task   Task   `json:"task"`   // параметры заданий
}

// ----------------------------------------------------------------------------
// CostsValues — сгенерированные длительности системных действий.
type CostsValues struct {
	Load       int
	Dispatch   int
	IOInit     int
	IOInterrupt int
	Terminate  int
}

// ----------------------------------------------------------------------------
// SystemValues — сгенерированные значения системных параметров.
type SystemValues struct {
	MemorySize    int
	TickDurationMs int
	IOMaxDuration  int
	Costs         CostsValues
}

// ----------------------------------------------------------------------------
// TaskValues — сгенерированные значения параметров задания.
type TaskValues struct {
	Size       int
	Commands   int
	IOPercent  int
	BasePriority int
}

// ----------------------------------------------------------------------------
// Default возвращает параметры по умолчанию. Они используются, если файл
// параметров не найден или не задан.
func Default() Set {
	return Set{
		Seed: DefaultSeed,
		System: System{
			MemorySize:     Param{Min: 4096, Max: 8192, Step: 1024, Comment: "Объём памяти, единиц"},
			TickDurationMs: Param{Min: 10, Max: 100, Step: 10, Comment: "Длительность такта, мс"},
			IOMaxDuration:  Param{Min: 1, Max: 10, Step: 1, Comment: "Максимальная длительность ВВ, тактов"},
			Costs: Costs{
				Load:       Param{Min: 1, Max: 1, Step: 1, Comment: "Затраты на загрузку задания, тактов"},
				Dispatch:   Param{Min: 1, Max: 1, Step: 1, Comment: "Затраты на выбор процесса, тактов"},
				IOInit:     Param{Min: 1, Max: 1, Step: 1, Comment: "Затраты на инициализацию ВВ, тактов"},
				IOInterrupt: Param{Min: 1, Max: 1, Step: 1, Comment: "Затраты на обслуживание прерывания ВВ, тактов"},
				Terminate:  Param{Min: 1, Max: 1, Step: 1, Comment: "Затраты на завершение задания, тактов"},
			},
		},
		Task: Task{
```

```
                        Size:        Param{Min: 100, Max: 500, Step: 50, Comment: "Размер задания, единиц
памяти"},
                        Commands:    Param{Min: 10, Max: 20, Step: 1, Comment: "Число команд задания"},
                        IOPercent:   Param{Min: 0, Max: 50, Step: 5, Comment: "Процент команд ВВ, %"},
                        BasePriority: Param{Min: 1, Max: 10, Step: 1, Comment: "Базовый приоритет
задания"},
                },
        }
}

// ----------------------------------------------------------------------------
// IsZero сообщает, что набор параметров не заполнен.
func (s Set) IsZero() bool {
        return s.System.MemorySize.Step == 0 &&
                s.System.TickDurationMs.Step == 0 &&
                s.System.IOMaxDuration.Step == 0 &&
                s.Task.Size.Step == 0
}

// ----------------------------------------------------------------------------
// Validate проверяет корректность всех диапазонов.
func (s Set) Validate() error {
        params := []struct {
                name string
                p    Param
        }{
                {"system.memorySize", s.System.MemorySize},
                {"system.tickDurationMs", s.System.TickDurationMs},
                {"system.ioMaxDuration", s.System.IOMaxDuration},
                {"system.costs.load", s.System.Costs.Load},
                {"system.costs.dispatch", s.System.Costs.Dispatch},
                {"system.costs.ioInit", s.System.Costs.IOInit},
                {"system.costs.ioInterrupt", s.System.Costs.IOInterrupt},
                {"system.costs.terminate", s.System.Costs.Terminate},
                {"task.size", s.Task.Size},
                {"task.commands", s.Task.Commands},
                {"task.ioPercent", s.Task.IOPercent},
                {"task.basePriority", s.Task.BasePriority},
        }

        for _, item := range params {
                if err := item.p.validate(item.name); err != nil {
                        return err
                }
        }

        if s.System.MemorySize.Min < 1 {
                return fmt.Errorf("system.memorySize: минимум должен быть не меньше 1")
        }
        if s.System.TickDurationMs.Min < 1 {
                return fmt.Errorf("system.tickDurationMs: минимум должен быть не меньше 1")
        }
        if s.System.IOMaxDuration.Min < 1 {
                return fmt.Errorf("system.ioMaxDuration: минимум должен быть не меньше 1")
        }
        if s.Task.Size.Min < 1 {
                return fmt.Errorf("task.size: минимум должен быть не меньше 1")
        }
        if s.Task.Commands.Min < 1 {
                return fmt.Errorf("task.commands: минимум должен быть не меньше 1")
        }
        if s.Task.IOPercent.Min < 0 || s.Task.IOPercent.Max > 100 {
```

```
		return fmt.Errorf("task.ioPercent: диапазон должен находиться в пределах 0..100")
	}
	if s.Task.BasePriority.Min < 1 {
		return fmt.Errorf("task.basePriority: минимум должен быть не меньше 1")
	}

	return nil
}

// ----------------------------------------------------------------------------
// validate проверяет один диапазон.
func (p Param) validate(name string) error {
	if p.Step <= 0 {
		return fmt.Errorf("%s: шаг должен быть больше 0", name)
	}
	if p.Min > p.Max {
		return fmt.Errorf("%s: минимум %d больше максимума %d", name, p.Min, p.Max)
	}
	return nil
}
```

internal/params/read.go

```
package params

import (
	"encoding/json"
	"fmt"
	"os"
)

// ----------------------------------------------------------------------------
// ReadFile читает JSON-файл параметров. Файл должен читаться один раз при
// запуске модели; повторное чтение при генерации заданий не выполняется.
func ReadFile(path string) (Set, error) {
	file, err := os.Open(path)
	if err != nil {
		return Set{}, fmt.Errorf("открыть файл параметров %q: %w", path, err)
	}
	defer file.Close()

	decoder := json.NewDecoder(file)
	decoder.DisallowUnknownFields()

	var set Set
	if err := decoder.Decode(&set); err != nil {
		return Set{}, fmt.Errorf("разобрать файл параметров %q: %w", path, err)
	}

	if set.Seed == 0 {
		set.Seed = DefaultSeed
	}
	if err := set.Validate(); err != nil {
		return Set{}, fmt.Errorf("некорректные параметры в %q: %w", path, err)
	}

	return set, nil
}
```

internal/params/generate.go

```
package params

import "math/rand/v2"
```

```
// ----------------------------------------------------------------------------
// Generate — подпрограмма «Генерация значения параметра».
// Возвращает случайное значение из диапазона [Min, Max] с шагом Step.
func (p Param) Generate(r *rand.Rand) int {
	if p.Step <= 0 || p.Max <= p.Min || r == nil {
		return p.Min
	}

	count := (p.Max-p.Min)/p.Step + 1
	if count <= 0 {
		return p.Min
	}
	return p.Min + r.IntN(count)*p.Step
}

// ----------------------------------------------------------------------------
// GenerateSystem — подпрограмма «Генерация параметров системы».
// Генерирует фактические значения системных параметров по диапазонам.
func (s Set) GenerateSystem(r *rand.Rand) SystemValues {
	return SystemValues{
		MemorySize:     s.System.MemorySize.Generate(r),
		TickDurationMs: s.System.TickDurationMs.Generate(r),
		IOMaxDuration:  s.System.IOMaxDuration.Generate(r),
		Costs: CostsValues{
			Load:       s.System.Costs.Load.Generate(r),
			Dispatch:   s.System.Costs.Dispatch.Generate(r),
			IOInit:     s.System.Costs.IOInit.Generate(r),
			IOInterrupt: s.System.Costs.IOInterrupt.Generate(r),
			Terminate:  s.System.Costs.Terminate.Generate(r),
		},
	}
}

// ----------------------------------------------------------------------------
// GenerateTask — обновлённая подпрограмма «Генерация нового задания».
// Генерирует параметры задания по диапазонам из файла.
func (t Task) GenerateTask(r *rand.Rand) TaskValues {
	return TaskValues{
		Size:         t.Size.Generate(r),
		Commands:     t.Commands.Generate(r),
		IOPercent:    t.IOPercent.Generate(r),
		BasePriority: t.BasePriority.Generate(r),
	}
}
```

5. Результаты проверки

Новые тесты не добавлялись. Проверка выполнялась ручными прогонами и существующими тестами проекта.

№	Проверка	Ожидаемый результат	Фактический результат	Вывод
1	Чтение params.json	12 диапазонов, seed 2026, статус «Загружен»	12 строк, seed 2026, params.json загружен	Выполнено
2	Отсутствующий файл	Ошибка чтения, значения по умолчанию	Ошибка открытия файла; используются params.Default()	Выполнено
3	Некорректный файл: min > max	Ошибка с именем параметра	system.memorySize: минимум 9000 больше максимума 100	Выполнено
4	Повторяемость: два запуска по 500 тактов	Одинаковые значения	generated=36, completed=32, ΣT_mono=868, история совпадает	Выполнено
5	Существующие тесты проекта	Все проходят	go test ./... — ok	Выполнено
6	Сборка Wails	Сборка успешна	wails build -tags webkit2_41 — Built	Выполнено

Вывод существующих тестов:

```
$ go test ./... -count=1
?   os_model                    [no test files]
?   os_model/cmd/osmodel        [no test files]
ok  os_model/internal/os        0.002s
?   os_model/internal/params    [no test files]
ok  os_model/internal/regulator 0.002s
ok  os_model/internal/stats     0.002s
```

При повторном запуске с тем же params.json и seed 2026 последовательность заданий, команд и статистика совпадают. Это подтверждает корректность единого генератора и отсутствие повторного чтения файла.

6. Экранные формы

На рисунке 6 показан главный экран модели после запуска с params.json. В примере использованы seed 2026, 66 загруженных и 64 завершённых задания, T_мод = 1000, T_сист = 860, D_multi = 105,3 %, D_sys = 46,2 %.

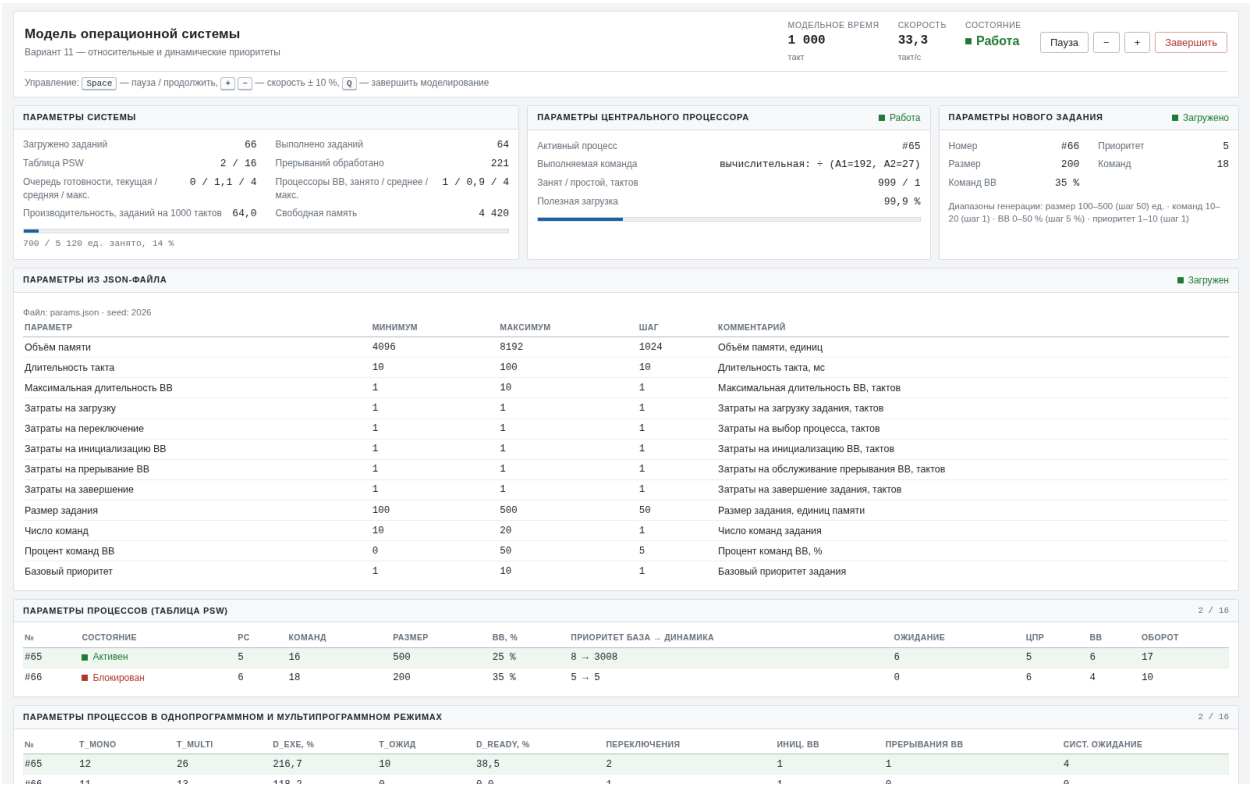


Рисунок 6 — Главный экран с параметрами из JSON-файла

В верхней части экрана отображаются сгенерированные системные параметры: объём памяти, скорость, длительность ВВ и затраты ОС. Под параметрами нового задания расположена панель «Параметры из JSON-файла».

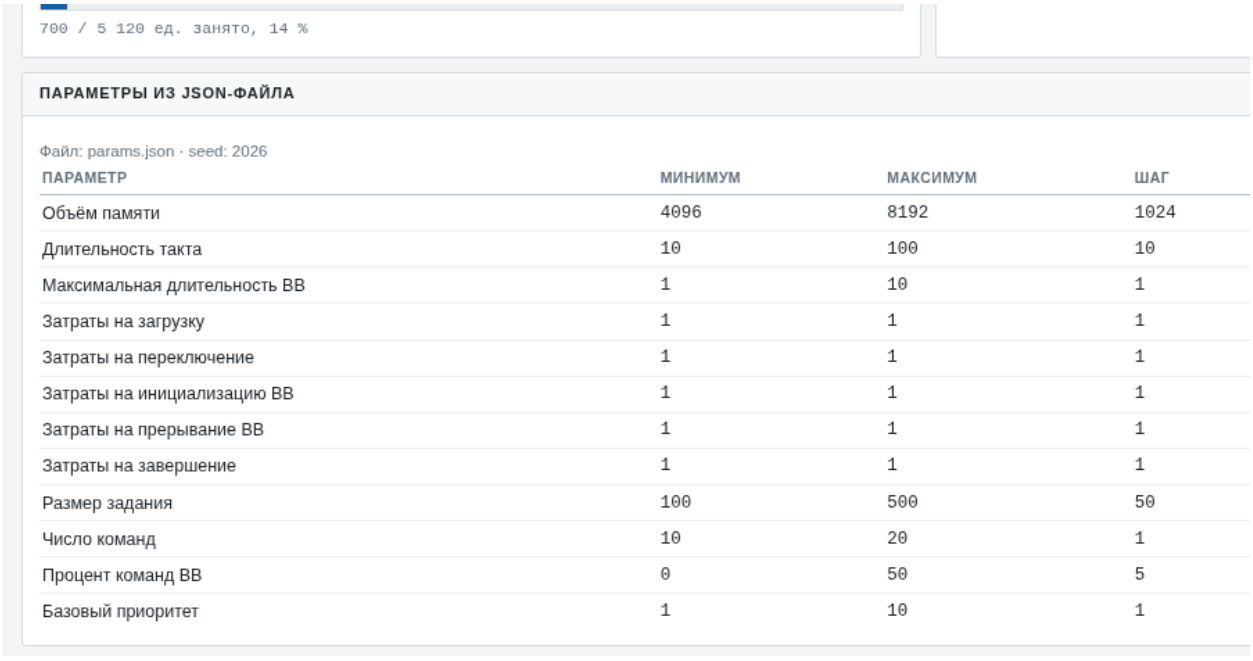


Рисунок 7 — Панель параметров из JSON-файла (фрагмент: имя, минимум, максимум и шаг)

Панель показывает имя файла, seed и таблицу всех параметров. Диапазоны и шаг из файла используются при генерации каждого нового задания; в «Параметрах нового задания» они выводятся вместе с дискретностью.

Выводы

В ходе лабораторной работы реализовано чтение входных параметров из JSON-файла params.json. Добавлены типы Param, System, Task и Set, подпрограммы ReadFile, Param.Generate, GenerateSystem и GenerateTask. Обновлена генерация нового задания: размер, число команд, процент BB и приоритет берутся из диапазонов файла.

Файл читается один раз при запуске. Единый генератор math/rand/v2 с фиксированным зерном обеспечивает повторяемость системных параметров, заданий, команд и длительностей BB. Проверены валидный файл, отсутствующий файл, некорректный диапазон и совпадение двух последовательных запусков. Существующие тесты проекта и сборка Wails проходят.